

Especificación técnica de requisitos

Librería extensible de optimización metaheurística en Rust

Autor

David Muñoz del Valle

Tutor

Gabriel Jesús Luque Polo



Índice

Índice	2
1. Alcance y objetivos generales	3
2. Requisitos funcionales	3
RF-01: Definición de la Función Objetivo:	3
RF-02: Modelado del Espacio de Búsqueda:	3
RF-03: Gestión de Restricciones:	3
RF-04: Validación Automática de Modelos:	4
RF-05: Selección de Metaheurísticas:	4
RF-06: Configuración de Hiperparámetros:	4
RF-07: Criterios de Parada Personalizables:	4
RF-08: Determinismo y Control de Semillas (Seeding):	4
RF-09: Implementación de funcionalidades multihilo:	4
RF-10: Inyección de Operadores Personalizados:	5
RF-11: Reporte del resultado:	5
RF-12: Sistema de Observabilidad (Telemetría):	5
RF-13: Generación de gráficos:	5
RF-15: Inserción de datos en problemas mediante ficheros de textos:	6
RF-16: Serializable:	6
3. Requisitos no funcionales	6
RNF-01: Desarrollo nativo en Rust:	6
RNF-02: Independencia de dependencias:	6
RNF-03: Documentación exhaustiva:	6
RNF-04: Arquitectura modular:	6
RNF-05: Simplicidad de uso:	6
RNF-06: Estabilidad numérica:	6
RNF-07: Eficiencia del software:	7
4. Casos de uso	7
CU1: Uso de la biblioteca original	7
CU2: Extensión de la biblioteca	8

1. Alcance y objetivos generales

La realización de este proyecto tiene como objetivo el desarrollo de una librería de optimización metaheurística extensible en Rust, diseñada bajo los principios de alto rendimiento y seguridad de memoria. Este proyecto busca paliar la fragmentación y escasez de herramientas maduras en el ecosistema de Rust, proporcionando una base sólida para investigadores y desarrolladores que necesitan una alternativa a los entornos tradicionales en Java, C++ o Python.

Para que podamos estar satisfechos con el resultado de la biblioteca, primero debemos entender cuáles son nuestros objetivos y hasta dónde queremos llegar.

En cuanto a nuestros objetivos principales, se encuentra, obtener un rendimiento, una velocidad de ejecución parecida a las implementaciones líderes de C++, que la biblioteca obtenga la abstracción suficiente como para que se pueda adaptar a los problemas definidos por los usuarios y que sea capaz de realizar mutaciones sobre las poblaciones en paralelo.

También es importante definir lo que está fuera del alcance del proyecto, por ejemplo, no se pretende realizar ningún tipo de interfaz gráfica, todo el funcionamiento se desarrollará mediante línea de comandos.

2. Requisitos funcionales

Definición y Modelado del Problema

RF-01: Definición de la Función Objetivo:

El sistema debe permitir al usuario definir una función de evaluación, ya sea para problemas de minimización o maximización.

RF-02: Modelado del Espacio de Búsqueda:

El sistema debe ser capaz de manejar diferentes tipos de variables: continuas (reales), discretas (enteros) y combinatorias (permutaciones).

RF-03: Gestión de Restricciones:

La biblioteca debe proveer mecanismos para manejar restricciones del problema.

RF-04: Validación Automática de Modelos:

El sistema debe poder verificar antes de ejecutar si el algoritmo está bien definido, teniendo en cuenta las variables y las restricciones definidas.

Ejecución y Control del Algoritmo

RF-05: Selección de Metaheurísticas:

El sistema debe permitir al usuario elegir entre un catálogo de algoritmos implementados (ej. Algoritmos Genéticos, Recocido Simulado, Enjambre de Partículas).

RF-06: Configuración de Hiperparámetros:

El sistema debe permitir la parametrización de cada algoritmo (ej. tamaño de población, tasa de mutación, temperatura inicial, número de iteraciones).

RF-07: Criterios de Parada Personalizables:

El usuario debe poder definir cuándo detener el algoritmo: por número máximo de iteraciones, por tiempo transcurrido, por alcanzar un valor de calidad objetivo o por estancamiento.

RF-08: Determinismo y Control de Semillas (Seeding):

Debido a la naturaleza de los algoritmos, es necesario introducir aleatoriedad en su ejecución. En este sistema, dicha aleatoriedad se implementa mediante generación pseudoaleatoria.

Con el fin de garantizar la reproducibilidad de los resultados, el sistema deberá permitir al usuario definir una semilla (seed) para el generador de números pseudoaleatorios. De este modo, utilizando la misma semilla y las mismas condiciones de entrada, el algoritmo producirá siempre los mismos resultados, facilitando así la validación, comparación y análisis experimental.

RF-09: Implementación de funcionalidades multihilo:

Los algoritmos serán dotados de herramientas para que puedan ejecutar puedan ejecutarse de manera paralela mediante varios hilos, manteniendo la seguridad en memoria.

Extensibilidad y Personalización

RF-10: Inyección de Operadores Personalizados:

El sistema debe permitir al usuario definir y utilizar sus propios operadores de variación (cruce, mutación, selección) sin modificar el núcleo de la biblioteca.

Salida y Análisis de Resultados

RF-11: Reporte del resultado:

Al finalizar la ejecución, el sistema debe entregar las soluciones encontradas y sus correspondientes valores de aptitud.

RF-12: Sistema de Observabilidad (Telemetría):

La biblioteca debe permitir inyectar "Observadores" para monitorizar en tiempo real la convergencia, la velocidad y la calidad de las soluciones obtenidas por los algoritmos.

RF-13: Generación de gráficos:

El sistema de observabilidad, debe poder crear gráficos fácilmente interpretables, que muestren la calidad de las soluciones obtenidas y la evolución de esta misma con cada iteración de los operadores durante la ejecución del algoritmo.

Otros

RF-14: Inclusión de pruebas:

Para asegurar la mantenibilidad y la calidad del código, se desarrollarán pruebas unitarias y de integración.

RF-15: Inserción de datos en problemas mediante ficheros de textos:

Los algoritmos podrán importar variables, restricciones y otros conjuntos de datos desde archivos CSV, JSON o YAML, lo que facilitará la definición y optimización de

problemas complejos mediante el uso de adaptadores compatibles con este formato.

RF-16: Serializable:

Todas las estructuras deben poder implementar los traits de *serde*, para que estas sean serializables.

3. Requisitos no funcionales

RNF-01: Desarrollo nativo en Rust:

La biblioteca debe estar desarrollada 100% en el lenguaje de programación Rust.

RNF-02: Independencia de dependencias:

La biblioteca no debe depender de crates externos (fuera de la estándar de Rust), *Zero-dependency policy*.

RNF-03: Documentación exhaustiva:

El código debe tener una documentación clara y concisa.

RNF-04: Arquitectura modular:

El diseño debe separar claramente los componentes del sistema, permitiendo intercambiarlos sin modificar el núcleo.

RNF-05: Simplicidad de uso:

Debe ser fácil e intuitiva la incorporación de nuevos problemas por parte del usuario.

RNF-06: Estabilidad numérica:

El sistema debe ser capaz de solventar los problemas de redondeo causados por los tipos decimales f32 y f64.

RNF-07: Eficiencia del software:

La biblioteca final debe ser capaz de realizar sus funcionalidades con una velocidad más que notable.

4. Casos de uso

Para la definición correcta de las secuencias a seguir por los actores en los diferentes casos de uso, necesitamos mostraros los diferentes comandos o líneas de código que serán utilizados.

Estos serán indicados con este formato: **este es el comando o el código**

CU1: Uso de la biblioteca original

CU1	Desarrollador quiere usar un algoritmo metaheurístico para optimizar la solución a un problema que tiene, pero no pretende desarrollarlo.	
Actores	Desarrollador	
Pre condiciones	1. El desarrollador tiene instalado el toolchain de Rust y Git en su máquina. 2. El desarrollador tiene un producto/proyecto inicializado en Rust. (Puedes inicializarlo con el siguiente comando) cargo new proyecto 3. Conoce la existencia del crate rmetal.	
Post condiciones	1. El algoritmo metaheurístico está integrado en el proyecto. 2. El proyecto compila correctamente. 3. El sistema es capaz de arrojar una solución optimizada al problema del desarrollador.	
Secuencia	#	Acción (Actor)
	1	Añade como dependencia el crate rmetal. cargo add metaheuristics-nature
	2	Importa la librería en su archivo fuente (ej. main.rs) use rmetal::*;
	3	Configura el tipo de problema y de algoritmo que más encaja con su caso.
	4	Modifica los parámetros de estos para su caso particular.
	5	Construye y ejecuta el proyecto

CU2: Extensión de la biblioteca

CU2	Desarrollador quiere implementar por su cuenta un algoritmo de optimización metaheurístico en Rust, pero no quiero empezar de 0, por tanto usa la biblioteca rmetal que es extensible.	
Actores	Desarrollador	
Pre condiciones	1. Tiene instalado el toolchain de Rust (Rustc, Cargo). 2. El desarrollador tiene un producto/proyecto inicializado en Rust. (Puedes inicializarlo con el siguiente comando) <code>cargo new proyecto</code> 3. Conoce la existencia del repositorio rmetal en GitHub.	
Post condiciones	1. El "Proyecto Principal" importa exitosamente la versión modificada de rmetal. 2. El proyecto compila y ejecuta el algoritmo personalizado correctamente. 3. (Opcional) El desarrollador está en posición de hacer un Pull Request si desea contribuir con su algoritmo a la librería original.	
Secuencia	#	Acción (Desarrollador)
	1	Realiza un "Fork" del repositorio original de rmetal en GitHub hacia su propia cuenta y lo clona en su equipo local. <code>git clone https://github.com/DRLKs/rmetal.git</code>
	2	Modifica directamente el código fuente de su clon de rmetal, agregando las estructuras y lógica matemática de su nuevo algoritmo metaheurístico.
	3	Abre su Proyecto Principal y edita el archivo de configuración (cargo.lock) para importar la versión modificada de rmetal directamente desde su repositorio de Git, en lugar del registro oficial o desde tu mismo sistema de archivos. Desde GitHub: <pre>[dependencies] # Opción A: A la rama principal por defecto rmetal = { git = "https://github.com/tu_usuario/rmetal.git" }</pre> <pre># Opción B: A una rama específica rmetal = { git = "https://github.com/tu_usuario/rmetal.git", branch = "dev" }</pre>

		<pre># Opción C: A un commit específico rmetal = { git = "https://github.com/tu_usuario/rmetal.git", rev = "a1b2c3d" }</pre> Desde tu sistema de archivos: <pre>[dependencies] rmetal = { path = "../proyectos/tu_fork" }</pre>
	4	En tu proyecto principal, importa la librería y utiliza el nuevo algoritmo que tú mismo desarrollaste en el paso 2.
	5	Construye y ejecuta el Proyecto Principal